DEAL ROOM V2

OWASP Top 10:2025 Security Assessment

Static source-code review | React frontend, Flask backend and project documentation

Assessment date: 5 November 2026
Assessment type: Manual static application security review
Framework: OWASP Top 10:2025
Scope: Three source/documentation archives supplied by the project owner.

Limitations: This is not a penetration test, DAST exercise, infrastructure assessment or formal certification. Findings are based on the supplied code and documentation. Exploitability, runtime behavior, live database settings, TLS termination and cloud controls have not been verified.

Executive Summary

Deal Room V2 has meaningful security foundations: server-side JWT validation, database-backed active-role checks, centralized authorization logic, role-aware resource checks, Marshmallow validation on several business write paths, SQLAlchemy ORM usage, password hashing, audit records and security-oriented tests.

The supplied code is **not ready for public production exposure**. The most urgent issue is the public first-user bootstrap: if the production database is empty, the first registrant is automatically approved and assigned Leadership. Other significant risks include predictable fallback signing secrets, debug mode defaulting to enabled in run.py, access and refresh tokens stored in localStorage, hard-coded development credentials in seed scripts, and no authentication rate limiting.

Severity summary

Severity	Count	Summary
Critical (conditional)	1	Public first signup can become Leadership if production DB is empty.
High	4	Fallback signing secrets, debug-on default, localStorage tokens, development seed credentials.
Medium	7	No auth throttling, exception detail leakage, headers/TLS evidence gap, CORS, supply chain, logging/alerting, auth input validation.
Low / assurance gap	2	Audit tamper-evidence gap and email enumeration.

Severity is a source-review prioritization, not a CVSS score. The critical rating is conditional on an empty production database with public signup enabled.

Immediate decision

Do not launch publicly until F-01 to F-06 are remediated and validated. At minimum: secure the first privileged account bootstrap, remove fallback secrets, disable debug by default, remove/gate seed credentials, add authentication throttling, and address browser-readable refresh tokens.

Reviewed inputs and limitations

The backend archive includes Flask application code, routes, services, models, migrations, tests, requirements files and environment files. The frontend archive includes src/ but not package.json or a package lockfile. requirements.txt pins backend package versions but no hash lockfile was supplied. No CI workflow, Docker, Terraform, Kubernetes, reverse-proxy or TLS configuration was present in the reviewed archives. The uploaded backend .env and .env.save files contain database connection configuration; values are intentionally not reproduced. Rotate any real or reused database credentials contained in those files.

Detailed Findings

F-01. Public first-user signup grants Leadership — CRITICAL (conditional)

OWASP: A06 Insecure Design; A01 Broken Access Control; A07 Authentication Failures

Evidence: backend/backend/app/api/auth_routes.py:13-20 exposes POST /api/auth/signup without authentication. backend/backend/app/services/auth_service.py:199-217 checks whether the users table is empty and, if so, creates the first account as APPROVED and assigns Leadership. AuthService._security_lock() (auth_service.py:39-58) serializes the decision but does not verify the registrant is the intended owner.

Impact: If the public service is exposed before a trusted owner is provisioned and the production database is empty, an internet user can register first and obtain the highest-privilege role. The lock prevents simultaneous signup races, not malicious first registration.

Remediation: Do not expose signup while production DB is empty. Provision the first Leadership identity through a one-time controlled operator action or protected bootstrap secret/allowlist. Disable bootstrap after use; optionally disable public signup. Add a test proving an untrusted first registrant cannot obtain Leadership.

F-02. Predictable fallback Flask and JWT signing secrets — HIGH

OWASP: A02 Security Misconfiguration; A04 Cryptographic Failures

Evidence: backend/backend/app/config/config.py:12-13 defines fixed SECRET_KEY and JWT_SECRET_KEY fallback values. backend/backend/app/__init__.py:44-48 only overrides JWT_SECRET_KEY when the environment variable exists.

Impact: If a deployment omits these environment variables, the app can start with source-known signing material. A known symmetric JWT signing key can undermine token authenticity and permit forged-token attacks.

Remediation: Fail closed in production: require strong independent secrets from a secret manager/environment and abort startup if absent, short or equal to known placeholders. Remove production fallbacks and rotate any keys used outside local development.

F-03. Flask debug mode defaults to enabled — HIGH

OWASP: A02 Security Misconfiguration

Evidence: backend/backend/run.py:7-13 defaults FLASK_DEBUG to '1'. Starting the service with python run.py and no explicit FLASK_DEBUG=0 enables debug mode. The supplied .env also contains development/debug settings; no production process manifest was supplied.

Impact: Flask's interactive debugger is unsafe for public exposure and can expose application internals or powerful debugging capabilities if reachable.

Remediation: Use Gunicorn or another production WSGI server. Default debug to false and fail startup if production is configured with debug enabled. Verify the actual deployment command rather than relying on developer environment variables.

F-04. JWT access and refresh tokens stored in localStorage — HIGH

OWASP: A07 Authentication Failures; A04 Cryptographic Failures

Evidence: frontend/src/auth/authStorage.js:1-65 stores both accessToken and refreshToken in localStorage. frontend/src/api/axiosClient.js reads them and attaches Bearer tokens to requests.

Impact: Any same-origin JavaScript execution caused by a current or future XSS flaw can read and exfiltrate both tokens. A stolen refresh token can extend the attacker’s access window. React's default escaping is beneficial, and no dangerouslySetInnerHTML/innerHTML sink was found in supplied src/, but localStorage remains a sensitive-token exposure.

Remediation: Prefer Secure, HttpOnly, SameSite cookies with CSRF protection and suitable refresh rotation/revocation. This requires backend/frontend changes because JWT location is currently headers-only. If retaining Bearer tokens, use short access lifetimes, stronger refresh-token protection, strict CSP and careful XSS prevention; document residual risk.

F-05. Hard-coded development credentials in seed scripts — HIGH (operationally conditional)

OWASP: A07 Authentication Failures; A08 Software or Data Integrity Failures

Evidence: backend/backend/app/seed/seed_admin.py:11-13 defines a fixed Leadership email/password and creates that account if absent (lines 24-31). backend/backend/create_v2_users.py:27 creates development users with a fixed test password. backend/backend/app/seed/seed_v2.py contains a development seed password constant.

Impact: If seed scripts are run against production, known credentials can create or expose privileged identities. The code describes seed_admin as development-oriented but does not enforce a production environment guard.

Remediation: Make every seed/reset script refuse APP_ENV=production. Remove fixed passwords; use one-time random credentials or operator-supplied secrets and require password reset. Separate development fixtures from production bootstrap/migrations.

F-06. No login/signup rate limiting or brute-force controls found — MEDIUM

OWASP: A07 Authentication Failures; A06 Insecure Design

Evidence: Public login/signup routes are in backend/backend/app/api/auth_routes.py:13-31. AuthService.login() verifies credentials in auth_service.py:229-248. No Flask-Limiter or equivalent throttling, account backoff/lockout, abuse challenge or authentication abuse control was found in supplied source.

Impact: Login is exposed to password guessing and credential stuffing. Signup can be abused for account spam. Password hashing alone does not provide abuse protection.

Remediation: Add shared IP- and account-keyed rate limits with progressive backoff, generic failure responses and alerts. Consider MFA for Leadership/Admin. Use a shared rate-limit backend such as Redis for multi-instance deployments.

F-07. Database exception details returned by stakeholder endpoint — MEDIUM

OWASP: A10 Mishandling of Exceptional Conditions; A09 Security Logging & Alerting Failures

Evidence: backend/backend/app/controllers/stakeholder_controller.py:44-73 prints tracebacks and returns detail fields containing SQLAlchemy exception text to clients for IntegrityError, SQLAlchemyError and generic exceptions.

Impact: Driver errors may reveal schema names, constraints, SQL fragments, database topology or user-provided values. Tracebacks can expose operational details in broadly accessible logs.

Remediation: Return generic stable error identifiers/messages. Keep detailed exceptions server-side with restricted access, correlation IDs and secret/PII redaction. Never return str(err), database orig values or tracebacks in API responses.

F-08. Production TLS and browser security headers not evidenced — MEDIUM (deployment-dependent)

OWASP: A02 Security Misconfiguration

Evidence: No response-header middleware or deployment proxy configuration for HSTS, CSP, X-Content-Type-Options, Referrer-Policy, frame protections or Permissions-Policy was found. No TLS termination/certificate/redirect configuration was supplied.

Impact: If neither application nor edge applies these controls, browser defense-in-depth against script injection, clickjacking, MIME sniffing and downgrade/mixed-content risks is weaker. Their absence in the archives does not prove the host does not provide them.

Remediation: Define production headers at trusted ingress/reverse proxy or Flask middleware. Enforce HTTPS; enable HSTS after HTTPS is stable; deploy a restrictive CSP, nosniff, appropriate Referrer-Policy and frame-ancestors/frame denial. Verify headers from the public endpoint.

F-09. CORS always includes localhost development origins — MEDIUM

OWASP: A02 Security Misconfiguration

Evidence: backend/backend/app/middleware/cors.py:7-22 reads CORS_ORIGINS but unconditionally adds http://localhost:5173 and http://127.0.0.1:5173; supports_credentials=True.

Impact: This unnecessarily expands production origin trust. With current header-based JWTs this is not proof of a direct token theft path, but it weakens environment separation and could matter more if cookie authentication is adopted.

Remediation: Use exact environment-specific production origins and do not append localhost in production. Disable credentialed CORS unless cookie auth requires it; if enabled, add CSRF protection and explicit origin tests.

F-10. Incomplete dependency inventory and no supply-chain scanning evidence — MEDIUM

OWASP: A03 Software Supply Chain Failures

Evidence: backend/backend/requirements.txt pins versions but has no hashes; requirements.in contains unpinned direct dependencies. Frontend archive lacks package.json and a JS lockfile. No Dependabot, pip-audit/OSV, CodeQL, Trivy, SBOM or image-signing workflow was supplied.

Impact: Complete frontend/backend dependency risk cannot be measured from these files. A pinned version is not proof that a package is free of known CVEs.

Remediation: Provide the real frontend manifest and lockfile. Generate SBOMs and add dependency/container scanning to CI. Update packages through reviewed PRs, use hash-locked builds where practical and pin release actions/images by immutable

digest. Run current CVE scans before deployment.

F-11. Security logging and alerting not production-ready — MEDIUM

OWASP: A09 Security Logging & Alerting Failures

Evidence: backend/backend/app/utils/logging.py:4-8 configures basic unstructured INFO logging. Debug print/traceback paths remain in stakeholder_controller.py:50-72 and backend/backend/app/api/poc_routes.py:74-80. AuditLog captures business events, but no security alerting or centralized log/retention configuration was supplied.

Impact: Repeated login failures, privilege changes, suspicious authorization failures and 5xx spikes may not be correlated or alerted. Debug output can expose identifiers and operational context.

Remediation: Adopt structured logs with request IDs, actor/role, route and outcome. Redact credentials, tokens, database URLs and PII. Alert on authentication abuse, role/admin changes, token anomalies and 4xx/5xx spikes. Define retention and access restrictions.

F-12. Authentication payload validation is weaker than business API validation — MEDIUM

OWASP: A10 Mishandling of Exceptional Conditions; A06 Insecure Design

Evidence: backend/backend/app/schemas/auth_schema.py defines SignupSchema/LoginSchema, but auth_routes.py calls AuthService directly with request.get_json(). AuthService.signup() only checks truthiness (auth_service.py:199-201); login indexes data['email'] and data['password'] directly (lines 229-231). No explicit password length/strength policy is applied in this path.

Impact: Malformed or non-object JSON and missing keys can trigger unexpected exceptions instead of consistent 4xx responses. Unbounded input lengths can waste resources. Signup validation is weaker than the schemas imply.

Remediation: Use the existing Marshmallow auth schemas at the route boundary. Set field length bounds and a documented password policy; handle malformed JSON and unexpected exceptions centrally without exposing internals.

F-13. Audit records are not shown to be tamper-evident — LOW / ASSURANCE GAP

OWASP: A08 Software or Data Integrity Failures; A09 Security Logging & Alerting Failures

Evidence: AuditLog and ActivityService.log() provide application-level audit inserts. No cryptographic chaining, independently controlled immutable store or database permission separation for audit rows was found in the supplied files.

Impact: A privileged DB operator or compromised DB credential may alter audit history outside the normal application service path. This is an assurance gap, not evidence that an ordinary API user can edit audit rows.

Remediation: For higher assurance, restrict the application DB role to insert-only on audit records where practical, export security events to independently controlled append-only storage and periodically verify integrity.

F-14. Signup response enables email enumeration — LOW

OWASP: A07 Authentication Failures

Evidence: backend/backend/app/services/auth_service.py:204-206 checks for an existing email and returns a distinct 'Email already exists' error to the unauthenticated signup caller.

Impact: An unauthenticated party can determine whether an email address has an account. Impact depends on whether membership is sensitive.

Remediation: Return neutral signup responses and communicate next steps through verified email. Rate-limit signup and avoid exposing account status.

OWASP Category Assessment

Category	Status	Summary
A01 Broken Access Control	PARTIAL / NOT CERTIFIED	Business API routes found in the route inventory use authentication/active-role decorators. AuthorizationService reloads status, auth_version and active-role membership from DB. Service-level object checks and tests exist. No confirmed bypass from static review. Direct IDOR/privilege-escalation tests remain unexecuted.
A02 Security Misconfiguration	FAIL	Fallback secrets, debug-on default, localhost CORS origins retained and no supplied evidence of production TLS/security headers.
A03 Software Supply Chain Failures	FAIL / INCOMPLETE	Frontend manifest/lockfile absent; backend requirements lack hashes; no automated dependency/SBOM/image scanning evidence.
A04 Cryptographic Failures	FAIL / PARTIAL	Fallback signing keys and browser-readable tokens are significant. Passwords are hashed using PBKDF2-SHA256, not stored plaintext. TLS and DB-at-rest encryption cannot be assessed.
A05 Injection	NO OBVIOUS STATIC FINDING	No obvious eval/exec, unsafe YAML load, shell execution or interpolated SQL injection found. Reviewed DB access uses SQLAlchemy ORM or bound parameters. Runtime fuzzing remains pending.
A06 Insecure Design	FAIL / PARTIAL	Public first-user bootstrap can assign Leadership to whoever registers first against an empty DB. Signup abuse controls are missing.
A07 Authentication Failures	FAIL	localStorage tokens, no rate limits, hard-coded seed credentials, no MFA evidence, weak signup validation and email enumeration.
A08 Software or Data Integrity Failures	PARTIAL / ASSURANCE GAP	Migrations, version checks and POC history exist. Production seed safeguards and tamper-evident audit storage are not evidenced.
A09 Security Logging & Alerting Failures	FAIL / PARTIAL	Business audit records exist, but logging is basic, debug output remains, and central security alerting/retention controls are not supplied.
A10 Mishandling of Exceptional Conditions	FAIL	Stakeholder endpoint returns raw database exception details; malformed auth payloads can cause unexpected errors; global safe exception handling is not evidenced.

Positive Security Controls Observed

• **Centralized server-side authorization:** AuthorizationService.current_context() reloads the user and validates active/revoked/approved status, auth_version, active role and current role membership.

• **Protected API surface:** discovered business endpoints use business_access_required or phase2_auth_required. Public signup/login are expected public entry points; root and health are informational.

• **Role-specific authorization:** service logic performs relationship/scope checks; React route guards are not the only security boundary.

• **Password hashing:** Werkzeug password hash helpers are used; no plaintext password persistence was observed.

• **SQL access:** reviewed queries predominantly use SQLAlchemy ORM; the raw SQL in create_v2_users.py uses named bind parameters.

• **Input validation:** Opportunity and stakeholder write paths use Marshmallow schemas; search bounds query length and page size.

• **Concurrency/audit foundations:** row-version checks, audit records and POC history exist.

• **PDF escaping:** POC report generation escapes ampersand and angle brackets before rendering user-controlled text.

Validation Performed / Not Performed

Performed: archive inventory; manual review of frontend auth storage and Axios interceptors, Flask configuration, route decorators, auth service, authorization layer, selected controllers/services, seed scripts, schemas, logging and dependency files; pattern checks for dangerous execution, raw SQL, unsafe DOM sinks, rate limiting and security headers; backend app source passed Python compileall syntax compilation.

Not performed: pytest, frontend production build, browser tests, live API tests, IDOR/privilege-escalation tests, brute-force, CSRF/XSS/injection fuzzing, TLS/header verification, PostgreSQL migration/concurrency tests, dependency CVE scanning, SBOM generation, Docker image scanning or cloud infrastructure assessment.

The analysis runtime did not have Flask and Marshmallow installed, so application tests could not be run. The supplied project documentation also says direct IDOR/privilege-escalation/concurrency certification requires the normal project environment and PostgreSQL. Frontend package.json and lockfile were absent from the frontend archive.

Prioritized Remediation Plan

P0 — Before any public exposure
Gate/remove public first-user Leadership bootstrap; require production secrets and fail startup if missing; force debug off and use Gunicorn; add production guards to seed scripts; rotate any real DB credentials present in the shared .env files.

P1 — Before production pilot
Add login/signup rate limiting; address localStorage refresh-token exposure; validate auth payloads/password lengths; remove raw exception details; define exact production CORS, TLS and browser security headers.

P2 — Before production sign-off
Provide frontend package manifest/lockfile; run current dependency CVE scans; add SBOM and CI security gates; improve structured security logging/alerting; verify backups and audit protections.

P3 — Runtime certification
Run role-by-role IDOR and privilege-escalation tests against PostgreSQL; test revoked/stale tokens; authentication abuse; XSS/CSRF/injection; public TLS/CORS/headers; full frontend/backend regression.

Recommended Runtime Test Matrix

Authentication: Repeated invalid logins; known/unknown email responses; malformed JSON; expired access/refresh tokens; refresh reuse; logout/revocation.

Authorization / IDOR: For each role, access another user's opportunity, POC, stakeholder, delivery project, activity and follow-up IDs; test read/update/delete and transitions.

Privilege escalation: Unassigned role selection; stale auth_version; revoked user; direct admin/Leadership endpoints; first signup on empty production-like DB.

Input handling: Oversized JSON; malformed types; unexpected fields; long strings; HTML/script in notes; SQL metacharacters; invalid external links.

Browser security: XSS token theft feasibility; CSP; CORS from untrusted origins; CSRF if cookies adopted; HTTPS/HSTS/clickjacking headers.

Operations: Missing production secrets; production seed-script invocation; DB outage; migration failure; log redaction; backup restore.

Final Assessment

Deal Room V2 has a stronger authorization and workflow foundation than a typical early-stage CRUD app. The urgent risks are not an obvious static SQL injection or a clearly unprotected business API route; they are privileged public bootstrap behavior, predictable fallback secrets, unsafe development defaults, browser-readable refresh tokens and development seed credentials.

Decision: continue in a controlled development/staging environment, but do not approve public production deployment until P0/P1 remediation and runtime security tests are complete. This is a baseline assessment, not a claim of OWASP compliance.

No secret values, passwords, tokens or database connection strings are reproduced in this report. Code paths are relative to the supplied archive roots.